Truly Automated Formal Specification and Verification with Near-Zero Human Labour

Don Syme
GitHub Next
United Kingdom
dsyme@github.com

LeanSquad
AI Automation Workflow (GitHub Agentic Workflows)
lean-squad@automated

Copilot CLI
AI Coding Agent (GitHub)

Claude Sonnet 4.6
AI Model (Anthropic)

ABSTRACT

Formal verification has historically demanded deep expertise in proof assistants and months of dedicated human effort, limiting its adoption to a handful of well-funded teams. We show an example of how LLM-driven automated agentic workflows can shift this cost nearly entirely to compute. Using *Lean Squad*, an open-source workflow that runs as a scheduled GitHub Actions job, we produce a machine-checked formal specification and verification of core safety properties of `raft-rs`—a production Rust implementation of the Raft distributed consensus protocol—with near-zero human labour. The human author required no expertise in Lean, formal methods, or proof engineering; his role was limited to designing the reusable workflow, merging pull requests, and writing four brief GitHub issue comments across 129 autonomous agent runs.

The agent autonomously researched the codebase, selected 26 verification targets, designed a layered proof architecture, wrote all specifications, and completed all proofs: 718 theorems across 79 Lean 4 files (stdlib only, no Mathlib), culminating in the end-to-end theorem `raftReachable_safe`—for any cluster state reachable via valid Raft transitions from an empty initial state, no two nodes ever apply different log entries at the same committed index—with zero `sorry`. All five abstract hypotheses of the `RaftReachable` model are fully discharged from concrete protocol theorems, leaving no axioms. One formulation bug was caught by the `sorry` discipline; no implementation bugs were found. An additional 28 correspondence-validation files provide 625 compile-time `#guard` assertions confirming that the Lean models agree with the Rust implementation on concrete inputs.

The total LLM token cost was approximately $400 (129 runs at ~$3 each using Claude Sonnet 4.6 pricing at time of publication). The Lean Squad workflow is open-source, installs on any GitHub repository with two commands, and requires only an LLM API token. The proof artefacts and full run history are available at `dsyme/raft-lean-squad`.

ACM Reference Format:
Don Syme, LeanSquad, Copilot CLI, and Claude Sonnet 4.6. 2026. Truly Automated Formal Specification and Verification with Near-Zero Human Labour. In . ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Note on authorship and voice. Throughout this paper, "we" generally reflects the perspective of *LeanSquad*, an autonomous AI automation workflow implemented using GitHub Agentic Workflows, with *Copilot CLI* (GitHub) as the underlying AI coding agent using the *Claude Sonnet 4.6* (Anthropic) LLM. The human author (Don Syme) took a deliberately *hands-off* approach: he designed the automated Lean Squad workflow itself (§3) and merged pull requests, but was not involved in target selection, the layered proof architecture, specification writing, or any actual proof engineering. His total intervention over the project's 129 runs amounted to four brief GitHub issue comments (§6.3).

1 INTRODUCTION

Distributed consensus algorithms are notoriously subtle. The Raft protocol was designed to be understandable [33], yet its correctness argument spans term management, log matching, leader completeness, and state-machine safety across multiple interacting invariants. Production implementations add further complexity: joint membership changes, in-flight

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

message tracking, flow control, and optimised quorum computation.

Formal verification of distributed systems is well-established but typically expensive: IronFleet [17] required person-years of Dafny proof engineering; Woos et al. [54] verified a simplified Raft in ~50,000 lines of Coq. The expertise barrier—fluency in a proof assistant, invariant discovery, and multi-month proof campaigns—has limited adoption to a handful of well-funded teams. Our contribution addresses this barrier directly, and is different in three ways:

(1) We target a *production* Raft implementation (`raft-rs`, used in TiKV [3]), not a pedagogical model.
(2) We use Lean 4 *stdlib only*—no Mathlib—providing a lightweight, self-contained verification artefact.
(3) The entire effort is *automated*: every target selection, specification, proof architecture decision, and proof was produced by an LLM-driven FV agent across 129 independent runs, with only four brief human interventions (§6.3). The human author required no Lean expertise, no formal-methods background, and invested near-zero hours of proof-engineering labour.

The top-level theorem (§5) is:

```
theorem raftReachable_safe [DecidableEq E]
    (hd : Nat) (tl : List Nat) (cs : ClusterState E)
    (hvoters : cs.voters = hd :: tl)
    (h : RaftReachable cs) : isClusterSafe cs
```

This is machine-checked by Lean 4 with zero `sorry`.

2 BACKGROUND

2.1 The Raft Consensus Protocol

Raft replicates a state machine through a distributed log. A cluster of n nodes elects a leader; the leader accepts log entries from clients and replicates them to followers via `AppendEntries` RPCs. An entry is *committed* once a majority quorum of nodes has acknowledged it. Safety requires that no two nodes ever apply different entries at the same log index.

The `raft-rs` codebase (derived from TiKV's raft-rs [3]) implements the Consensus Module in Rust: 52 source files covering quorum arithmetic, log management, flow control, membership change, and configuration validation. It supports joint consensus for membership transitions, a ring-buffer-based in-flight message tracker, and per-node progress tracking.

2.2 Lean 4

Lean 4 [8] is a dependently typed proof assistant and programming language. Proofs are written in a tactic language with automation including `omega` (linear arithmetic), `simp` (rewriting), and `decide`/`native_decide` (decidable propositions). We use Lean 4.30.0-rc2, stdlib only. All proofs are machine-checked by the Lean kernel; the `sorry` keyword marks unproved obligations.

3 THE LEAN SQUAD WORKFLOW

The Lean Squad workflow [1] is a GitHub Agentic Workflow—a declarative specification that orchestrates an LLM coding agent (e.g. Copilot CLI or Codex CLI) as a scheduled GitHub Actions job. It was designed by the human author and open-sourced as a reusable workflow template; it is not specific to Raft or to this repository. Once installed, the workflow runs autonomously on a schedule (every 8 hours in our deployment), with no human in the loop for any individual run.

3.1 Design Principles

Four design principles underpin the workflow:

Zero human involvement necessary. The agent handles the entire FV effort: research, target selection, proof architecture design, specification writing, and proof engineering. The human need only merge PRs and occasionally nudge the process. This shifts the cost of formal verification from scarce human proof-engineering expertise to LLM compute, dramatically lowering the barrier to entry and making FV a practical option for any codebase—not just those with dedicated verification teams.

Optimistic. The agent does not wait for perfect conditions. It starts where the code is and finds what can be verified, even if that means working with approximations or partial proofs.

Incremental. Each run contributes one small piece. A spec with `sorry` today becomes a proof next week. A research note today drives a formal spec next month. Persistent memory ensures each run builds on the last.

Findings-first. A counterexample is a success, not a failure. Finding a bug through formal verification is exactly the point.

Transparent. Every PR, issue, and comment includes an AI disclosure. Specs document their approximations. Proofs document what is still open. The history of every run is public, and the full history of the specifications and proofs is available in the repository for the human and the agent to examine as needed.

3.2 Phase-Weighted Task Selection

Each run begins with a deterministic pre-step that inspects the repository's FV artefacts (Lean files, spec documents, open issues and PRs) and computes a phase-weighted probability distribution over eleven tasks. Two tasks are drawn and communicated to the agent; the agent confirms them against

its persistent memory and executes them. A mandatory final task (status-issue update) always runs.

The eleven tasks, in order of the typical progression, are:

(1) **Research & Target Identification.** Survey the codebase to identify FV-amenable functions, data structures, and algorithms. Document expected benefit, spec size, proof tractability, and recommended approach. Produces `RESEARCH.md` and `TARGETS.md`. If a critique document exists, incorporate its feedback to adjust priorities.
(2) **Informal Spec Extraction.** For the highest-priority unstarted target, extract a precise informal specification by reading the code and inferring the design intention—preconditions, postconditions, invariants, edge cases, concrete examples, and inferred intent not explicit in the code. Flags ambiguities for maintainer review.
(3) **Formal Spec Writing (Lean 4).** Write Lean 4 type definitions, function stubs, and key `theorem` declarations with `sorry` as proof bodies. Must pass `lake build` before opening a PR.
(4) **Implementation Extraction.** Translate the relevant Rust implementation into Lean 4 functional definitions so propositions can be formally connected to actual code. For imperative code, create a pure functional model and document the abstraction.
(5) **Proof Assistance.** Attempt to prove stated propositions using Lean 4 tactics. When a proof obligation cannot be closed, investigate whether the spec or the implementation is wrong. If a counterexample is found, file a bug issue.
(6) **Correspondence Review.** Review how faithfully each Lean model corresponds to the Rust source. Assess correspondence level (exact, abstraction, approximation, or mismatch) and document all divergences.
(7) **Proof Utility Critique.** Honestly assess whether the FV work is useful—are the proved properties meaningful, at the right level of abstraction, and likely to catch real bugs? Identify highest-value gaps.
(8) **Correspondence Validation.** Write compile-time `#guard` tests that evaluate the Lean model on concrete inputs and check the result against the Rust implementation. Optionally use Aeneas for mechanical extraction; otherwise write manual test fixtures with matching Rust `#[test]` functions.
(9) **CI Automation.** Set up and maintain `lean-ci.yml` to automatically verify Lean proofs on every PR and push.
(10) **Project Report.** Create and maintain `REPORT.md`—a comprehensive report with mermaid diagrams summarising the entire FV effort.
(11) **Conference Paper.** Write and maintain a LaTeX paper under `formal-verification/paper/`, compile to PDF, and commit to the repository.

The weighting scheme adapts automatically to the current state of the repository. When no FV work exists, Task 1 (Research) dominates. Once research is done, Task 2 rises. As informal specs accumulate, Task 3 gains weight, and so on up through proofs and beyond to reflective tasks (critique, correspondence review, paper writing). Table 1 illustrates the weight progression.

Task	Name	Weight (early)	Weight (late)
1	Research	10.0	2.0
2	Informal Spec	8.0	2.0
3	Formal Spec	8.0	2.0
4	Implementation	6.0	2.0
5	Proof Assistance	12.0	3.0
6	Correspondence Review	10.0	3.0
7	Critique	4.0/10.0	2.0/3.0
8	Corr. Validation	12.0	5.0/3.0
9	CI	12.0	2.0
10	Report	3.0/8.0	2.0/3.0
11	Paper	2.0/8.0	1.0/3.0

Table 1: Phase-weighted task selection (11 tasks). "Early" weights apply when the task's prerequisites are met but the task has not yet been completed; "late" weights apply once the artefact exists. Slash notation (e.g. 4.0/10.0) indicates the weight while proofs are incomplete vs. after proof maturity. Tasks 5, 8, and 9 are most heavily weighted during active proof work.

3.3 Persistent Memory and Cumulative Progress

A critical design element is persistent repo-memory. At the start of every run, the agent reads its memory to recover the identified FV targets, their current phases (1–5), key choices, open questions, bugs found, and ideas to try. At the end of every run, the agent updates memory with new discoveries, proved theorems, and remaining obligations.

Memory may be stale (referencing merged PRs or closed issues), so the agent is instructed to verify references against the current repository state. This mechanism enables cumulative progress across independent runs: the layered architecture, the choice of targets, and the proof strategy all emerged organically from the agent's interaction with its own memory and the growing codebase, not from human direction.

3.4 Hard Requirements

The workflow enforces several hard requirements to maintain integrity:

- The Lean toolchain **must** be successfully installed before any `.lean` files are written or submitted. If installation fails, Tasks 3–5 are skipped entirely for that run.
- Every PR that includes `.lean` files must include a verification status block: either "`lake build` passed" or an explicit disclosure that proofs are unverified.
- The agent must never claim proofs are verified unless `lake build` actually passed.
- All PRs, issues, and comments must include an AI transparency disclosure.

3.5 Artefacts Produced

The workflow produces a rich set of artefacts beyond Lean proofs: `RESEARCH.md` (FV survey and rationale), `TARGETS.md` (prioritised target list with phase status), per-target informal specs, `CORRESPONDENCE.md` (Lean-to-Rust mapping), `CRITIQUE.md` (proof utility assessment), `REPORT.md` (comprehensive project report), and the conference paper itself. A continuously-updated GitHub issue serves as a rolling dashboard showing the status of every target.

3.6 Installation and Usage

The Lean Squad workflow is open-source and can be installed on any GitHub repository with two commands. A token for an LLM coding agent is required; at the time of writing, Copilot CLI (GitHub), Codex CLI (OpenAI), and Claude Code (Anthropic) are supported as underlying agents.

```
# Install the GitHub Agentic Workflows CLI extension
gh extension install github/gh-aw

# Add Lean Squad to the current repository
gh aw add-wizard githubnext/agentics/lean-squad
```

The `add-wizard` command walks the user through configuration: selecting the schedule (default: every 8 hours), choosing the agent provider, and storing the API token as a GitHub Actions secret. Once committed, the workflow runs autonomously on the configured schedule with no further setup.

With just these two commands, any repository can start an automated formal verification effort with zero human specification or proof-engineering expertise required. Whether that is useful or not depends on the codebase and the agent's performance, but the barrier to entry is minimal. Getting started with formal verification of real software has never been easier.

4 METHODOLOGY

4.1 Target Selection

In its very first run (PR #93), the agent surveyed the `raft-rs` codebase via Task 1 (Research) and identified an initial set of **10 FV targets**, prioritised by tractability: `limit_size`, `Config::validate`, `vote_result`, `committed_index`, `find_conflict`, `maybe_append`, joint-config variants, `inflights`, and `progress`. That same run also produced the first informal specification (`limit_size`). The criteria were: pure functions with clear pre/postconditions, data-structure invariants, and proof tractability. No human input was involved in selecting or prioritising targets.

Notably, the first run's research plan proposed Lean 4 + Mathlib and flagged the Aeneas extraction toolchain as a possible route; both choices were later revised by the agent itself—switching to stdlib-only and dropping Aeneas—as it gained experience with the proof artefacts across subsequent runs.

Over later iterations the agent expanded the target list to **26 targets** and organised them into seven layers (Table 2). Additional targets were added as the agent discovered cross-module composition opportunities, protocol-level safety properties, a reachability model, and a complete election model. This layered architecture emerged from the agent's own analysis of module dependencies—it was not prescribed by the human author. Layers build on each other: Layer 2 (Quorum Arithmetic) uses types from Layer 1; Layer 4 (Composition) spans Layer 2 and Layer 3; Layers 5–6 depend on all preceding layers; Layer 7 (Election Model) provides concrete discharge lemmas for the abstract step hypotheses of Layer 6.

4.2 Specification Strategy

Each target was formalised in two phases. First, an *informal specification* was extracted by reading the Rust source, its tests, and documentation comments. This produced a prose document identifying preconditions, postconditions, invariants, edge cases, and open questions.

Second, the informal spec drove a *Lean 4 formalisation*: type definitions mirroring the Rust types (with deliberate approximations documented in CORRESPONDENCE.md), function stubs, and theorem statements. Proofs were then attempted incrementally, from simplest (structural induction, `omega`) to most complex (multi-lemma composition).

4.3 Modelling Choices

The Lean model deliberately abstracts several Rust features:

Type abstraction. `u64` voter IDs and index values are modelled as `Nat`. This eliminates overflow concerns (safe: no

Layer	Description	Files	Theorems
Core proof architecture (Layers 1–7):			
1	Data Structures	6	161
2	Quorum Arithmetic	8	135
3	Protocol Operations	3	43
4	Cross-Module Composition	3	26
5	Raft Safety	2	18
6	Reachability	2	11
7	Election Model (complete)	12	152
Correspondence validation (Layer 8):			
8	Correspondence Validation	28	2+625g
Additional module verifications (Layers 9–20):			
9	ReadOnly Operations	1	14
10	FindConflictByTerm	1	10
11	Flow Control	1	18
12	Configuration Invariants	1	11
13	Persistent-Storage Bridge	2	16
14	Progress Tracking	1	26
15	Raft-Log Append	1	16
16	Next-Entries Slicing	2	21
17	Progress Set	1	8
18	MemStorage Core	1	14
19	IsContinuousEnts	1	8
20	Restore	1	8
	Total	**79**	**718+625g**

Table 2: Proof layers and theorem counts. Layers 1–7 form the core safety-proof chain; Layers 9–20 are standalone module verifications. Layer 8 files contain 625 `#guard` assertions (g) evaluated at compile time; they contribute 2 named theorems (in `FindConflictCorrespondence.lean`). File count includes `Basic.lean` (helper definitions, 0 theorems).

practical Raft deployment approaches 2^{64}) and enables `omega` arithmetic.

Collection abstraction. `HashSet<u64>` voter sets are modelled as `List Nat`. Theorems that require uniqueness carry an explicit `voters.Nodup` hypothesis; those that do not hold for arbitrary lists.

Log abstraction. The per-node log is modelled as `VoterLogs E := Nat → Nat → Option E`—a pure function from (voter, index) to optional entry. This abstracts over the Rust `Vec`-based log structure while preserving the read semantics.

Protocol abstraction. Concrete Raft message types are modelled abstractly: `ValidAEStep` captures AppendEntries effects, `ElectionEpoch` captures election outcomes, and `BroadcastSeq` captures broadcast rounds. Term management and leader election are formalised at the level needed to discharge the `RaftReachable.step` hypotheses (Layer 7), but the full message-passing network model (UDP delivery, reordering, partitions) is not included.

Size semantics. For `limit_size`, protobuf message size is abstracted as an arbitrary `size : α → Nat` function. Proofs hold for any such function, making them strictly more general than the Rust implementation.

4.4 Proof Approach

We used the following tactic strategy, from simplest to most complex:

- `omega`: linear integer/natural arithmetic (most common)
- `simp`/`simp only`: definitional unfolding and rewriting
- `by_cases`: boolean and decidable case splits
- `induction`/`cases`: structural recursion on lists and inductives
- `exact`/`apply`/`refine`: explicit term construction
- `funext`: function extensionality

We used no `native_decide`, no axioms beyond Lean's kernel, and no `sorry` in the final proof state.

4.5 Model Validation (Layer 8)

A proof that *the Lean model* is correct conveys confidence only if the Lean model faithfully captures the Rust implementation. To ground the abstract models, we developed 26 *correspondence-test* files (Layer 8) containing compile-time `#guard` assertions. Each file re-evaluates the Lean model function on concrete inputs and checks the result against the expected value from the Rust implementation. Because `#guard` is evaluated by `lake build` at compile time (not at runtime), these are not optional tests—they are compile-time obligations that must pass for the build to succeed.

For each target, a matching Rust function (prefixed `test_. . . _correspondence`) exercises the Rust implementation on the same concrete inputs, providing independent confirmation. Together, the 625 Lean `#guard` assertions and 18+ Rust test functions provide two-sided behavioural evidence: both the Lean model and the Rust implementation produce identical outputs on the verified test cases.

5 RESULTS

5.1 Layer-by-Layer Proof Inventory

Layer 1: Data Structures. Six files prove correctness of individual Raft data structures in isolation.

`LimitSize.lean` (17 theorems) formalises the Rust function `util::limit_size`, which truncates a vector of protobuf messages to stay within a byte budget. Key results: *prefix correctness* (the result is a prefix of the input), *non-emptiness*

($\geq$ 1 entry always preserved), and *maximality* (the longest prefix within budget).

`Inflights.lean` (40 theorems) is the largest single file. It formalises the ring-buffer-based in-flight message tracker (`inflights.rs`), proving the abstract/concrete bridge: the efficient ring buffer and a simple list model compute identical `free_to` results.

`Progress.lean` (36 theorems) formalises the per-node progress tracker state machine, proving that the well-foundedness invariant (`matched + 1 ≤ next_idx`) is preserved across all transitions.

Layer 2: Quorum Arithmetic. Eight files prove the mathematical foundations of Raft safety. The cornerstone is `quorum_intersection_mem` (HQ20, `HasQuorum.lean`):

```
theorem quorum_intersection_mem
  (voters : List Nat) (hne : voters ≠ [])
  (a b : Nat → Bool)
  (ha : hasQuorum voters a = true)
  (hb : hasQuorum voters b = true) :
  ∃ v, v ∈ voters ∧ a v = true ∧ b v = true
```

This states: any two majority quorums over the same voter list share at least one concrete member. This is the quorum-intersection property that makes Raft (and Paxos) safe. The proof proceeds by an arithmetic argument using inclusion-exclusion and `omega`.

`CommittedIndex.lean` (17 theorems) formalises the *sort-then-index* algorithm that computes the committed index from a list of per-voter match indices, proving both safety (result is attested by a majority) and maximality (result is the largest such index).

Layer 3: Protocol Operations. Three files prove correctness of core Raft log operations. `MaybeAppend.lean` (18 theorems) gives a complete post-condition for `maybe_append`: the prefix is untouched and the suffix is correctly applied, providing a full behavioural specification of AppendEntries log modification.

`FindConflict.lean` (12 theorems) proves that the function returns zero iff all entries match, and returns the first mismatch index otherwise.

`MaybePersist.lean` (13 theorems) formalises the async-persist safety guards, proving a complete iff-characterisation of the three guards (monotonicity, firstUpdateIndex bound, term-match) and idempotence (a second call with the same arguments always returns false).

Layer 4: Cross-Module Composition. Three files prove properties spanning multiple independent modules. The key result is that if two voters have committed different entries at the same index, they must share a witness voter in their respective quorums—but the same voter cannot have voted for two different entries, giving a contradiction. A second file proves that `maybe_append` never advances a committed index beyond what the current quorum has certified.

Layer 5: Raft Safety. `RaftSafety.lean` (10 theorems) lifts quorum-level safety to the log-entry level. The central theorem is RSS1:

```
theorem raft_state_machine_safety
  [DecidableEq E]
  (voters : List Nat) (hne : voters ≠ [])
  (logs : VoterLogs E) (k : Nat)
  (e₁ e₂ : E)
  (h1 : isQuorumCommitted voters logs k e₁)
  (h2 : isQuorumCommitted voters logs k e₂)
  : e₁ = e₂
```

If two entries are simultaneously quorum-committed at the same index, they must be identical. The proof uses `quorum_intersection_mem` (HQ20) to obtain a shared witness voter and conclude that both entries equal the witness's log entry.

RSS8 provides the conditional end-to-end result: given the `CommitCertInvariant`—every applied entry is quorum-certified—the cluster is safe.

`RaftProtocol.lean` (8 theorems) proves that AppendEntries preserves the Log Matching Invariant and the No-Rollback property, formalising the Raft protocol transition rules at the log level.

```
inductive RaftReachable [DecidableEq E] :
  ClusterState E → Prop where
| init (voters : List Nat) :
    RaftReachable (initialCluster voters)
| step {cs cs' : ClusterState E}
    (hreach : RaftReachable cs)
    (hvoters : cs.voters = cs'.voters)
    (v : Nat)
    (hlogs' : ∀ w, w ≠ v →
      cs'.logs w = cs.logs w)
    (hno_overwrite : ∀ k,
      cs.committed v ≥ k →
      cs'.logs v k = cs.logs v k)
    (hqc_preserved : ∀ k e,
      isQuorumCommitted
        cs.voters cs.logs k e →
      ∀ w, cs'.logs w k = cs.logs w k)
    (hcommitted_mono : ∀ w,
      cs'.committed w ≥ cs.committed w)
    (hnew_cert : ∀ w k e,
      cs'.committed w ≥ k →
      cs.committed w < k →
      cs'.logs w k = some e →
      isQuorumCommitted
        cs'.voters cs'.logs k e) :
    RaftReachable cs'
```

Figure 1: The `RaftReachable` inductive protocol model.

Layer 6: Reachability. `RaftTrace.lean` (4 theorems) closes the gap between the conditional safety result (RSS8) and an unconditional statement. It defines the `RaftReachable` inductive predicate (Figure 1) and proves by structural induction that every reachable state satisfies

`CommitCertInvariant` (RT1), from which RSS8 yields the top-level theorem RT2:

```
theorem raftReachable_safe [DecidableEq E]
    (hd : Nat) (tl : List Nat) (cs : ClusterState E)
    (hvoters : cs.voters = hd :: tl)
    (h : RaftReachable cs) : isClusterSafe cs :=
  raft_end_to_end_safety_full hd tl cs hvoters
    (raftReachable_cci cs h)
```

The two-line proof follows immediately from RT1 and RSS8.

`MultiStepReachability.lean` (7 theorems) extends this to n-step chains: theorem MS2 provides an end-to-end safety certificate showing that a valid n-step AE sequence from a `RaftReachable` state yields another `RaftReachable` state satisfying the safety invariant.

Layer 7: Election Model (complete). Twelve additional files formalise Raft's election and commit machinery at a more concrete level, providing the theorems that fully discharge the `RaftReachable.step` hypotheses.

`RaftElection.lean` (53 theorems) formalises abstract Raft-step properties including single-voter log-change invariants, providing lemmas used by higher layers.

`LeaderCompleteness.lean` (10 theorems) formalises leader completeness properties needed to discharge `hqc_preserved`: the property that a newly elected leader has all quorum-certified entries in its log.

`ConcreteTransitions.lean` (8 theorems) formalises the `AppendEntries` effect function directly, proving CT1–CT5b: key properties of how the log is updated by a concrete AE step.

`CommitRule.lean` (9 theorems) formalises the Raft commit rule, proving that `leaderCommit` reflects a genuine quorum acknowledgement and *closing* the `hnew_cert` hypothesis of `RaftReachable.step`.

`MaybeCommit.lean` (12 theorems) formalises the commit-index advance function and proves *term safety*: the commit index advances only when the entry at the new committed index has the leader's current term, formalising Raft §5.4.2. This is the formal proof of the mechanism that prevents the "figure 8" problem from the Raft paper.

`ConcreteProtocolStep.lean` (14 theorems) introduces the `ValidAEStep` structure and proves the *A5 bridge*: if a state is `RaftReachable` and a valid AppendEntries step fires, the resulting state is also `RaftReachable`. This is the first theorem that directly connects a concrete Raft message to the abstract reachability model.

Four additional files close the `hqc_preserved` gap (the hardest hypothesis) through a chain: election safety (≤ 1 leader per term) $\rightarrow$ vote-granting implies `isUpToDate` $\rightarrow$ broadcast-sequence structural induction $\rightarrow$ `hqc_preserved` from broadcast $\rightarrow$ election lifecycle bridge.

`ElectionLifecycle.lean` (7 theorems) is the capstone file. It defines `ElectionEpoch`—a structure tying an election winner to a specific term and broadcast round—and proves that any `RaftReachable` state followed by an `ElectionEpoch` and a `ValidAEStep` produces a new `RaftReachable` state. `fullProtocolStep_safe` is the end-to-end result: no abstract axioms remain.

Two additional files formalise raft-log append (prefix preservation, committed-index monotonicity) and the `firstUpdateIndex` derivation for async-persist safety (the key invariant ensuring `maybe_persist` cannot advance past stable storage).

Layer 8: Correspondence Validation. Twenty-eight correspondence-test files provide compile-time evidence that the Lean models agree with the Rust implementation on concrete inputs. Each file evaluates the Lean model function on concrete inputs and checks the result against the expected value from the Rust implementation via `#guard` assertions, which are evaluated by `lake build` at compile time. Matching Rust test functions run the same cases via `cargo test`. In total: 625 compile-time correspondence assertions across 28 files (12 at "exact" correspondence, 16 at "abstraction" level) and 18+ Rust test functions. The full per-file breakdown is available in the repository's `REPORT.md`.

Layers 9–20: Standalone Module Verifications. Twelve additional layers (170 theorems across 14 files) verify individual Raft subsystems that do not feed into the safety chain: read-only request handling, backward-scan conflict resolution, flow-control byte accounting, configuration invariants, persistent-storage bridging, progress tracking, raft-log append, next-entries slicing, composed progress sets, in-memory storage, batch concatenation validity, and snapshot restoration. Each file is fully proved with 0 `sorry`; Table 2 gives per-layer file and theorem counts.

5.2 Bugs and Findings

No implementation bugs found. All 718 theorems are consistent with the Rust implementation, as further confirmed by 625 compile-time `#guard` correspondence assertions and 18+ Rust test functions.

Formulation bug caught and corrected. Early versions of `log_matching_property` (RSS3) and `raft_committed_no_rollback` (RSS4) claimed properties for *arbitrary* log states—with no hypotheses about how the logs were constructed. These statements are provably false (a log with random contents satisfies no log-matching property). The `sorry` mechanism acted as a "needs review" marker: since `sorry` was required but no proof could be found, the error was caught before it entered the proof base.

The corrected versions carry explicit hypotheses: `LogMatchingInvariantFor` and `NoRollbackInvariantFor`, which are exactly the invariants preserved by valid Raft transitions. Both were subsequently proved (run r130).

6 DISCUSSION

6.1 Proof Architecture and the Closed Protocol Gap

The `RaftReachable` model (Figure 1) provides honest accounting of what the safety proof depends on. The five `step` hypotheses (`hlogs'`, `hno_overwrite`, `hqc_preserved`, `hcommitted_mono`, `hnew_cert`) precisely state what a protocol transition must satisfy to preserve `CommitCertInvariant`.

As of Run 126, **all five hypotheses have been fully discharged** from concrete proved theorems:

- `hlogs'`: discharged by `ValidAEStep`, which models single-voter AppendEntries.
- `hno_overwrite`: discharged via the committed-≤-prev precondition and the concrete-transitions bridge.
- `hqc_preserved`: discharged by the election lifecycle chain, which combines election safety, the broadcast induction, and the election lifecycle bridge.
- `hcommitted_mono`: discharged from `ValidAEStep`.
- `hnew_cert`: discharged by the commit rule together with term safety (the commit index advances only for entries from the current leader's term).

The key theorem is `fullProtocolStep_safe`: given any `RaftReachable` state, an `ElectionEpoch` (election + broadcast), and a subsequent `ValidAEStep`, the resulting state is cluster-safe. No abstract axioms remain.

The proof chain for the hardest hypothesis (`hqc_preserved`) proceeds: election safety (at most one leader per term) → vote-granting implies `isUpToDate` → `BroadcastSeq` induction → `hqc_preserved` from broadcast → election lifecycle bridge.

Concretely, `hno_overwrite` corresponds to the Rust panic guard:

```
if conflict != 0
   && conflict <= self.committed {
  panic!("entry␣{}␣conflict
␣␣␣␣with␣committed␣log", conflict);
}
```

This panic is proved to be unreachable in a correct Raft execution by `hno_overwrite`. The `RaftProtocol` layer proves that `maybe_append` satisfies `hno_overwrite` given the appropriate preconditions.

6.2 Proof Utility Assessment

The 718 theorems divide into two groups by structural role. **The safety chain** (25 files, 341 theorems) forms a connected dependency graph from quorum arithmetic through to `fullProtocolStep_safe`, the end-to-end safety result. Every theorem in this chain has a genuine transitive dependency on the lower layers; removing any would break the final result. **Standalone module verifications** (25 files, 375 theorems) prove properties of individual Raft subsystems—inflights, log-unstable, progress tracking, read-only, flow control, etc.—that do not feed into the safety chain but independently validate critical data-structure invariants. Both groups contribute to overall confidence.

In particular:

- The quorum intersection theorem is the *only* mathematical fact about quorums used in the safety proof. If it were wrong, the proof would fail. Its proof is nontrivial (inclusion-exclusion + arithmetic), giving meaningful assurance.
- `MaybeAppend` gives a complete behavioural characterisation of AppendEntries log modification—the most frequent Raft protocol operation. A bug in `maybe_append` would cause one of these theorems to fail.
- `SafetyComposition` is the first theorem that cannot be stated within a single module; it requires both quorum arithmetic and committed-index computation (both Layer 2) to interact.
- The standalone modules, while not in the safety chain, verify the data-structure invariants that the Raft implementation relies on in practice. For example, the 40 `Inflights` theorems prove that the ring-buffer implementation computes identical results to a simple list model—a property that matters for operational correctness even though the safety proof abstracts over it.

Why standalone components are standalone. The 25 standalone files are not isolated by accident. `fullProtocolStep_safe` proves that no two nodes apply different entries at the same committed index—a property that depends only on quorum intersection, log-matching preservation, and the commit rule. It does *not* depend on how the log is stored (`Inflights`, `LogUnstable`, `MemStorage`), how entries are delivered to the application (`NextEntries`), flow control (`UncommittedState`, `ProgressTracker`), or read-only request handling (`ReadOnly`). Three groups could, however, strengthen the safety chain: (1) the *persistence chain* (45 theorems) could yield a crash-recovery safety theorem if `RaftReachable` were extended with a crash/restore transition; (2) `CommitRule` + `MaybeCommit` + `CrossModuleComposition` (28 theorems) prove the same

properties the main chain assumes via `ValidAEStep` fields, but from the Rust function models—linking them would give a concrete function-level discharge, reducing the trust boundary; (3) `TallyVotes` + `JointTally` + `JointVote` (67 theorems) could derive election uniqueness from the concrete vote-counting algorithm, strengthening `RE5` from an assumed property of `ElectionEpoch` to a proved consequence of the tally logic. None of these extensions would change the *statement* of EL7, but they would narrow the gap between the abstract model and the Rust code.

The main remaining abstraction is the Rust implementation itself: the Lean models capture the algorithms faithfully (as confirmed by 625 `#guard` assertions), but do not use mechanical extraction from Rust source. Fully unconditional safety with respect to the Rust binary would require mechanical extraction (e.g. via Aeneas [18]) or a verified compiler toolchain.

6.3 Automation and Human Intervention

The entire FV effort was conducted by the *Lean Squad* automated agent, an LLM-driven agent that runs as a GitHub Actions workflow (§3). Each run: reads persistent memory, draws two tasks from a phase-weighted schedule, installs the Lean toolchain, executes the tasks (writing or modifying Lean files, specs, or documentation), runs `lake build`, and creates a pull request with verified code. A final status-issue update always runs.

The human author's role was deliberately hands-off: he designed the Lean Squad workflow itself (§3), installed it on the repository, and reviewed merged PRs. Over the entire 129-run project, his total intervention consisted of four brief GitHub issue comments:

Intervention 1 (Issue #94 comment, run ≈28). After 370 theorems were proved but no state-machine-level safety theorem existed, the human wrote:

```
/lean-squad Address these concerns from
critique.md, plan out rest of work, write
theorems even if unproven, get the project
on Trajectory to completion
```

This triggered `RaftSafety.lean` and `CrossModuleComposition.lean` (run r128), adding 25 new theorems including the first end-to-end cluster safety results.

Intervention 2 (Issue #144, run ≈29). After the initial proof effort was declared complete, the human wrote:

```
/lean-squad Now FV is complete, please
update critique, correspondence etc. and
write a final REPORT.md project report.
Include mermaid diagrams
```

This produced the project report and updated documentation artefacts.

Intervention 3 (Issue #149, run ≈35). The most substantive intervention: the human pasted a detailed external critique identifying the structural gap in the `RaftReachable.step` hypotheses (specifically that `hqc_preserved` required leader completeness, which required an election model not yet formalised) and wrote:

```
/lean-squad Continue the work, adjusting
report, targets and replanning all future
work, based on this critique
```

This triggered the agent to incorporate the gap analysis and replan, eventually producing the election model (Layer 7), concrete protocol bridge, and leader completeness proofs.

Intervention 4 (Issue #255, run ≈125). The human asked the agent to assess composition completeness and then wrote:

```
/lean-squad OK go through and make progress
on whatever else needs doing to close out
the work
```

This directly produced `ElectionLifecycle.lean`, closing the final gap and fully discharging all five `RaftReachable.step` hypotheses.

What is remarkable. is not that four interventions occurred, but what did *not* require human intervention: the choice of all 26 FV targets, the layered proof architecture, all 25 informal specifications, all Lean formalisations, all 718 proofs, the 28 correspondence-validation files, the proof utility critique, the project report, and even an initial draft of this paper were produced autonomously. The human never selected a target, wrote a specification, suggested a proof strategy, or debugged a failing proof.

Across 129 runs spanning ≈18 weeks, the agent produced:

- 25 informal specification documents
- 79 Lean 4 source files (≈21,000 lines)
- 718 machine-checked theorems
- 625 compile-time `#guard` correspondence assertions
- 18+ Rust correspondence test functions
- 1 formulation bug caught and corrected
- 0 implementation bugs
- All 5 `RaftReachable.step` hypotheses fully discharged

The key insight enabling this level of autonomy is the workflow's phase-weighted task selection (§3.2): the agent always has a well-defined next action, and persistent memory ensures each run builds on a growing base of named lemmas and established infrastructure. The layered architecture emerged organically from the agent's interaction with its own memory and the growing codebase.

6.4 Lean Stdlib vs. Mathlib

We deliberately chose Lean 4 stdlib only, without Mathlib. This keeps the proof artefact self-contained and avoids

version-pinning issues with Mathlib's rapid development. The cost is the absence of mature list-processing lemmas and richer automation.

In practice, most proofs required only `omega`, `simp`, `by_cases`, and structural induction—all available in stdlib. The only Lean 4 stdlib pitfalls encountered were: `lemma` not being a top-level keyword (use `theorem` instead), `split_ifs` and `push_neg` not being available (use `by_cases` and `omega`), and `le_rfl` not existing (use `Nat.le_refl _`).

6.5 Lessons Learned

Phase-weighted task selection enables autonomy. The workflow's eleven-task structure with adaptive weights means the agent always has a well-defined next action. The agent does not need a human to tell it what to do next—the weight computation, combined with persistent memory, provides sufficient direction. The layered proof architecture emerged organically from this process rather than being prescribed. The correspondence-validation layer (Layer 8) emerged when the agent recognised the need to ground abstract Lean models in concrete Rust behavior, and the election-lifecycle layer emerged when the agent set about closing the `RaftReachable.step` hypotheses.

Persistent memory is essential for cumulative progress. An LLM agent can prove theorems in a new module if all dependencies are already available as named lemmas. Persistent memory ensures the agent knows which lemmas exist, which targets are complete, and what strategies have worked or failed in prior runs.

The `sorry` *discipline catches formulation bugs.* Proving with `sorry` as a placeholder and then trying to remove it is a reliable method for finding specification errors. Both RSS3 and RSS4 were caught this way.

One module per run, not the whole codebase. Each run focused on a single target. This made the Lean context small enough for the LLM to reason about correctly, and produced clean, reviewable pull requests.

Formal models must be honest about what they omit. The CORRESPONDENCE.md document records all known divergences between Lean models and Rust source. Making these explicit—rather than claiming full correspondence—is essential for correctly assessing proof value.

Negative results have value. Finding no implementation bugs in 718 theorem-checks is itself evidence of implementation quality. The absence of counterexamples in the verified paths raises confidence, even without a fully concrete model.

Not all theorems feed the top-level result, and that is fine. Only 25 of the 50 proof files (341 of 716 proof-file theorems) form the dependency chain to `fullProtocolStep_safe`. The remaining 25 files (375 theorems) verify standalone data-structure invariants. An LLM agent exploring a codebase will naturally verify what it encounters, producing useful module-level proofs even when they do not connect to the global safety theorem. This mirrors human FV practice: not every invariant proved in seL4 feeds the top-level kernel safety property. The standalone proofs still have value as independently verified specifications.

6.6 Cost

Each of the 129 Lean Squad runs consumed approximately $3 of LLM token cost at Claude Sonnet 4.6 pricing (at time of publication), for a total of approximately $400. This covers the full pipeline: reading persistent memory, researching the codebase, selecting tasks, installing the Lean toolchain, writing or modifying Lean files, running `lake build`, and creating pull requests. The GitHub Actions compute time (Linux runners) is negligible relative to token cost.

For comparison, IronFleet [17] required person-years of expert proof engineering in Dafny; Woos et al. [54] reported ~50,000 lines of Coq proof. Even valuing human proof-engineer time conservatively, the dollar cost of comparable manual efforts exceeds the LLM cost by orders of magnitude. The trade-off is clear: LLM-driven FV substitutes cheap, parallelisable compute for scarce, expensive human expertise. As model prices continue to fall, the economics will only improve.

6.7 Training Data Overlap

It is virtually certain that Claude Sonnet 4.6 was trained on published papers, source code, and proof artefacts related to the formal verification of Raft—including Ongaro's dissertation [32], the Verdi/VRaft Coq development [54], TLA+ specifications, and the `raft-rs` source code itself. The degree to which this prior exposure influenced the agent's target selection, proof architecture, and tactic choices cannot easily be measured.

We note two mitigating factors. First, the proofs are machine-checked by Lean's kernel: regardless of how the agent arrived at a proof term, its correctness is independently verified. Training-data influence cannot produce a false positive. Second, several architectural decisions—stdlib-only (no Mathlib), the layered decomposition, the `RaftReachable` abstract model—have no direct precedent in the prior Raft verification literature, suggesting that the agent did more than reproduce memorised material. Nevertheless, this overlap should be borne in mind when assessing the generalisability of our results to domains with less existing formal-verification literature.

7 RELATED WORK

We situate our contribution with respect to six threads of prior work.

7.1 Formal Verification of Raft

The closest predecessor is Woos et al. [54], who gave the first machine-checked proof of state-machine safety for Raft in Coq using the Verdi framework [53]. Their proof discovered 90 system invariants and extracted a runnable OCaml implementation. A key methodological contribution was *planning for change*: factoring proofs so that strengthening one invariant does not cascade failures throughout the development. Our work differs in three respects: (i) we target a *production* Rust implementation (`raft-rs`) rather than an extracted model; (ii) we use Lean 4 with stdlib only; and (iii) the entire effort is LLM-automated. Our layered architecture echoes their information-hiding methodology, but arises from the need to give the LLM agent a well-defined proof frontier rather than from human proof-maintenance concerns.

Subsequent Raft verification efforts have used diverse formalisms. Evrard [13] modelled Raft in LNT for model-checking with CADP. Bao et al. [4] and Ishibashi and Ogata [22] model-checked Raft leader election in NuSMV and Maude respectively. Sanabria et al. [42] verified Raft leader election safety using the Athena proof system. Bora et al. [7] modelled Raft in mCRL2, enabling behavioural equivalence checking. Xiong et al. [56] formalised split, merge, and membership changes for Raft (ReCraft), complementing our focus on log safety. None of these target a production implementation or produce end-to-end safety proofs at the level of `raftReachable_safe`.

Ongaro's TLA+ specification [32] provides a complementary model-checking approach. TLA+ can exhaustively check finite instances but does not produce machine-checked proofs over unbounded state spaces. Our Lean proofs hold for arbitrary cluster sizes and unbounded execution traces, within the `RaftReachable` model.

7.2 Verified Distributed Systems

IronFleet [17] verified Paxos-based replication and a distributed key-value store in Dafny with TLA-style temporal reasoning. The Disel framework [44] introduced dependent separation protocols for modular distributed-system verification in Coq. Sprenger et al. [46] proposed Igloo, combining compositional refinement with separation logic for distributed-system verification. Pîrlea and Sergey [35] mechanised blockchain consensus in Coq, demonstrating that protocol-level safety proofs are tractable in proof assistants.

More recently, Honoré et al. developed a series of verified distributed-object frameworks: Adore [21] for certified reconfiguration, ADO [19] for fault-aware compositional verification, and AdoB [20] for bridging benign and Byzantine consensus. Qiu et al. verified linearisable Byzantine distributed objects (LiDO [37]) and extended this to DAG-based consensus (LiDO-DAG [38]). Kim et al. [26] presented VeriRT, an end-to-end verification framework for real-time distributed systems. Lewchenko et al. [30] verified bolt-on strong consistency specifications. Zhao et al. [62] developed compositional verification of composite Byzantine protocols. Zhang et al. [60] studied inductive invariants for distributed protocols with a focus on proof ergonomics.

Two recent systems-level efforts are particularly relevant. AutoMan [61] combines automatic code generation with manual optimisations for verified distributed-system development, achieving strong guarantees on production-grade systems. Veil [34] provides a framework for automated and interactive verification of transition systems, offering an alternative path to the reachability-style reasoning in our Layer 6.

Blockchain consensus verification has also grown rapidly: Thomsen and Spitters [49] formalised Nakamoto-style proof of stake in Coq; Rashid et al. [39] verified safety properties of Ethereum 2.0 Beacon Chain epoch processing; Schultz et al. [43] verified a distributed dynamic reconfiguration protocol.

7.3 Formal Verification of Rust

Verifying Rust programs is an active area with several complementary approaches. RustBelt [25] established the semantic foundations of Rust's type system safety guarantees in Coq using Iris separation logic. Prusti [2] provides automated deductive verification for Rust via Viper, targeting ownership and memory safety. Creusot [10] translates Rust to WhyML for verification with Why3, handling borrows and ownership. Verus [28] embeds verification conditions directly in Rust using linear ghost types and SMT solving.

Our approach is orthogonal: rather than verifying the Rust code directly, we build a pure functional Lean model of the relevant algorithms and verify properties of that model, documenting the correspondence to Rust in `CORRESPONDENCE.md`. This is a deliberate trade-off: we lose mechanical connection to the Rust source (which Aeneas [18] could provide) but gain the full power of Lean's dependent types and tactic framework for protocol-level reasoning that SMT-based tools struggle with.

Bílý et al. [6] proposed a refinement methodology for distributed Rust programs that could, in principle, bridge our abstract Lean models with the concrete `raft-rs` code. Combining their refinement approach with our verified safety properties is a promising direction for future work.

7.4 LLM-Based Theorem Proving

Recent years have seen rapid progress in applying LLMs to formal theorem proving. LeanDojo [58] introduced a benchmark and retrieval-augmented approach for Lean proving, where an LLM retrieves relevant premises from Mathlib before generating tactic steps. LEGO-Prover [52] grows a reusable library of verified lemmas across proof attempts, conceptually similar to our persistent memory mechanism. Baldur [14] demonstrated whole-proof generation and repair with LLMs for Isabelle/HOL. Draft-Sketch-Prove [23] uses informal proofs as sketches that guide formal proof search. Copra [48] employs an LLM agent with backtracking for Lean theorem proving. Polu and Sutskever [36] showed that curriculum learning over formal mathematics enables LLMs to solve increasingly difficult problems. Song et al. [45] explored LLMs as interactive copilots for Lean users.

Bayazıt et al. [5] conducted a case study on the effectiveness of LLMs for verification with proof assistants, finding that LLMs can discharge simple lemmas but struggle with multi-step proofs requiring creative insight. Our experience *diverges*: the Lean Squad agent completed all 718 theorems including multi-step compositional proofs (Layers 4–7) that require chaining lemmas across modules. We attribute this difference to four factors: (i) the workflow's phase-weighted task selection (§3) decomposes the problem into tractable increments, each with a well-defined interface to the next; (ii) persistent memory across runs enables cumulative progress where each run builds on a growing base of named lemmas and established infrastructure; (iii) the remarkable improvement in raw LLM reasoning power in the intervening period—frontier models in 2025–2026 are substantially more capable at multi-step logical reasoning than those available during the Bayazıt et al. study; and (iv) the planning and tool-use capabilities of modern coding agents (Copilot CLI, Codex CLI), which provide structured multi-step reasoning, file navigation, and iterative error correction that go well beyond single-prompt LLM interactions.

These systems differ from our approach in a fundamental way: they target *benchmark-style* theorem proving, where the goal is to close a given lemma statement in an existing formalisation. The human provides the theorem statement and the library context; the LLM provides the proof. In contrast, *Lean Squad operates at the project level*: it selects what to verify, writes the specifications, builds the library from scratch, and proves the theorems—all within a production codebase that has no prior formalisation.

The closest analogue to our persistent-memory mechanism is LEGO-Prover's growing library [52], but LEGO-Prover operates within a single proof session, whereas Lean Squad accumulates knowledge across 129 independent GitHub Actions runs spanning weeks. AlphaProof [50] demonstrated that reinforcement learning over formal proofs can achieve competition-level mathematical reasoning, but targets Olympiad geometry rather than software verification, and requires massive compute for training.

Yao et al. [59] demonstrated mostly-automated verification of liveness properties using ranking functions, and Gopinathan et al. [15] developed mostly-automated proof repair for verified libraries—both pointing toward increasing automation in formal verification. Díaz et al. [11] introduced incremental certified programming, which shares our goal of making cumulative progress on large proof developments.

7.5 Autonomous AI Agents for Software Engineering

SWE-bench [24] established a benchmark for evaluating LLM agents on real-world GitHub issues. SWE-agent [57] demonstrated that purpose-built agent-computer interfaces improve automated issue resolution. Agentless [55] showed that simpler, non-agentic approaches can also resolve a significant fraction of issues.

These agents operate in a *reactive* mode: given a bug report or feature request, they produce a patch. Lean Squad operates in a *proactive, scheduled* mode: it runs every 8 hours regardless of external triggers, inspects the repository state, and advances the project. This distinction is important because formal verification is not naturally triggered by external events—it is a long-running engineering effort that requires sustained, cumulative progress.

The phase-weighted task selection mechanism (§3.2) can be viewed as a form of *automated project management*: the workflow maintains a notion of project phase, allocates effort accordingly, and adapts as the project matures. This is, to our knowledge, novel among LLM-agent architectures.

7.6 Proof Engineering and Large-Scale Verification

Ringer et al. [40] surveyed the landscape of formally verified software engineering, identifying proof maintenance as a central challenge—the same challenge that motivated the "planning for change" methodology of Woos et al. [54]. Dejon et al. [9] provided a systematic guide to proof engineering, and Luan et al. [31] studied compatibility issues across theorem prover versions, a practical concern we sidestep by using Lean 4 stdlib only. The proof-repair line of work by Ringer et al. [41] and Viola et al. [51] addresses proof maintenance across type equivalences, complementing our approach of isolating change behind layer boundaries.

Fiat-Crypto [12] demonstrated automatic synthesis of verified cryptographic arithmetic. The Aeneas toolchain [18, 47] enables mechanical Lean extraction from safe Rust; we note its applicability but do not use it in this work.

To calibrate expectations, we compare the scale of our verification with landmark projects. seL4 [27] verified a production OS microkernel: 8,700 lines of C, ∼200,000 lines of Isabelle/HOL proof, 11 person-years. CertiKOS [16] verified a concurrent OS kernel with 6,500 lines of C and ∼100,000 lines of Coq. CompCert [29] verified an optimising C compiler over ∼100,000 lines of Coq. IronFleet [17] verified Paxos replication with ∼42,000 lines of Dafny proof. Woos et al. [54] verified simplified Raft with ∼50,000 lines of Coq across 90 invariants.

Our ≈21,000 lines of Lean and 718 theorems are modest by comparison. However, the relevant metric is not raw scale but *effort*: those projects required person-years of expert proof engineering, while ours required zero human proof engineering and approximately 129 LLM agent runs (each ∼15–30 minutes of wall-clock time) plus two one-line human comments. If the approach scales—and the successful progression from Layer 1 through Layer 20 suggests it can—the implications for the cost of formal verification are significant.

Our contribution is, to our knowledge, the first machine-checked safety proof for a *production* Raft implementation, using a modern proof assistant without external libraries, and generated entirely by an automated LLM agent.

8 CONCLUSION

Formal verification has historically required deep expertise in proof assistants and months of dedicated human effort. This paper demonstrates that LLM-driven agents can eliminate that barrier.

Starting from zero, the Lean Squad agent—running as a scheduled GitHub Actions job with no human in the loop—autonomously selected 26 verification targets in a production Raft implementation, designed a layered proof architecture, wrote all specifications and proofs, and produced 718 machine-checked theorems across 79 files in 129 runs, culminating in the end-to-end safety theorem `raftReachable_safe` with zero `sorry` and all five abstract `RaftReachable.step` hypotheses fully discharged from concrete protocol theorems, plus the A5 bridge connecting concrete AppendEntries steps to the abstract reachability model, and an additional 28 correspondence-validation files providing 625 compile-time `#guard` assertions and 18+ Rust test functions confirming that the Lean models agree with the Rust implementation on concrete inputs. The human author's total involvement was: designing the reusable workflow, merging PRs, and writing four brief GitHub issue comments. He required no Lean expertise, no formal-methods training, and invested near-zero hours of proof-engineering labour.

The key enablers are: (i) a phase-weighted task selection mechanism that gives the agent a well-defined next action at every stage of the project; (ii) persistent memory that accumulates proved lemmas, target status, and proof strategies across independent runs; and (iii) the dramatically improved reasoning capabilities of modern coding agents (Copilot CLI with Claude Sonnet 4.6 in this project).

The proof architecture is layered, honest, and extensible: each layer's assumptions are explicit, and all five `RaftReachable.step` hypotheses have been discharged from concrete protocol theorems. The Lean Squad workflow is open-source and installs on any GitHub repository with two commands (§3.6), requiring only an LLM API token. We believe this shifts the economics of formal verification: the cost moves from scarce human proof-engineering expertise to commodity LLM compute, making machine-checked safety proofs a practical option for ordinary software teams.

Future work on Lean Squad includes reliably applying the Aeneas [18] toolchain to mechanically extract Lean models from safe Rust; generalising it to use a broader range of formal verification tools and methodology; and applying Lean Squad to other production codebases to test generality.

ACKNOWLEDGMENTS

This paper was co-authored by Don Syme, LeanSquad (an autonomous AI automation workflow using GitHub Agentic Workflows), Copilot CLI (GitHub's AI coding agent), and Claude Sonnet 4.6 (Anthropic's LLM). Don Syme designed the Lean Squad workflow and made four brief interventions; all 26 target selections, all specifications, all 718 proofs, the layered architecture, and the correspondence-validation harness were produced autonomously by the AI agent. All theorems are machine-checked by Lean 4.30.0-rc2 with zero `sorry`*, and all five* `RaftReachable.step` *hypotheses have been fully discharged. The proof artefacts, agent configuration, and full run history are available in the repository* `dsyme/raft-lean-squad`*. The Lean Squad workflow is open-source at* `githubnext/agentics/workflows/lean-squad.md`*.*

REFERENCES

[1] Lean Squad (Automated Agent). 2026. Lean Squad: Incremental Automated Formal Verification via LLM-Driven Proof Agents. In *Workshop on AI for Formal Methods.* Automated verification of raft-rs using Lean 4.

[2] Vytautas Astrauskas, Aurel Bílenère, Christoph Matheja, Federico Poli, Peter Müller, and Alexander J. Summers. 2022. The Prusti Project: Formal Verification for Rust. In *NASA Formal Methods Symposium.*

[3] TiKV Project Authors. 2024. raft-rs: The Raft Distributed Consensus Protocol in Rust. https://github.com/tikv/raft-rs

[4] Qianli Bao, Bo Li, Tong Hu, and Decheng Cao. 2022. Model Checking the Safety of Raft Leader Election Algorithm. In *QRS.* 400–409. https://doi.org/10.1109/QRS57517.2022.00048

[5] Birkan Bayazıt, Yixuan Li, and Xujie Si. 2025. A Case Study on the Effectiveness of LLMs in Verification with Proof Assistants. In *LMPL.* 91–105. https://doi.org/10.1145/3759425.3763391

[6] Adam Bílý, João Pereira, and Peter Müller. 2025. A Refinement Methodology for Distributed Programs in Rust. *OOPSLA* 9 (2025), 1921–1948. https://doi.org/10.1145/3763119

[7] Paul Bora, Phuc Minh, and Tim Willemse. 2024. Modelling the Raft Distributed Consensus Protocol in mCRL2. *Electronic Proceedings in Theoretical Computer Science* 399 (2024), 7–20. https://doi.org/10.4204/EPTCS.399.4

[8] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28*. Springer, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37

[9] Nicolas Dejon, Christophe Gaber, Gilles Grimaud, and Naim Jomaa. 2024. Code to Qed, the Project Manager's Guide to Proof Engineering. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–50. https://doi.org/10.1145/3664807

[10] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *ICFEM*.

[11] Tomas Díaz, Kenji Maillard, Nicolas Tabareau, and Éric Tanter. 2025. Incremental Certified Programming. *OOPSLA* 9 (2025), 499–526. https://doi.org/10.1145/3763068

[12] Andres Erbsen, Jade Philipoom, Jason Gross, Robert Sloan, and Adam Chlipala. 2019. Simple High-Level Code For Cryptographic Arithmetic With Effective Formal Verification. In *IEEE S&P*. https://doi.org/10.1109/SP.2019.00005

[13] Hugues Evrard. 2020. Modeling the Raft Distributed Consensus Protocol in LNT. *Electronic Proceedings in Theoretical Computer Science* 316 (2020), 15–39. https://doi.org/10.4204/EPTCS.316.2

[14] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *FSE*.

[15] Kiran Gopinathan, Mayank Keoliya, and Ilya Sergey. 2023. Mostly Automated Proof Repair for Verified Libraries. *PLDI* 7 (2023), 25–49. https://doi.org/10.1145/3591221

[16] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Jialin) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. 2016. CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels. In *OSDI*.

[17] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *SOSP*. ACM. https://doi.org/10.1145/2815400.2815428

[18] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. In *ICFP*. ACM. https://doi.org/10.1145/3547647

[19] Wolf Honoré, Joonseung Kim, Jieung Shin, and Zhong Shao. 2021. Much ADO About Failures: A Fault-Aware Model for Compositional Verification of Strongly Consistent Distributed Systems. *OOPSLA* 5 (2021), 1–31. https://doi.org/10.1145/3485474

[20] Wolf Honoré, Lun Qiu, Yoonseung Kim, Jieung Shin, Joonseung Kim, and Zhong Shao. 2024. AdoB: Bridging Benign and Byzantine Consensus with Atomic Distributed Objects. *OOPSLA* 8 (2024), 419–448. https://doi.org/10.1145/3649826

[21] Wolf Honoré, Jieung Shin, Joonseung Kim, and Zhong Shao. 2022. Adore: Atomic Distributed Objects with Certified Reconfiguration. In *PLDI*. 379–394. https://doi.org/10.1145/3519939.3523444

[22] Tsubasa Ishibashi and Kazuhiro Ogata. 2023. Formal Specification and Model Checking of Raft Leader Election in Maude. In *ICSCA*. 41–45. https://doi.org/10.1145/3587828.3587835

[23] Albert Q. Jiang, Sean Welleck, Jesse Michael Han, et al. 2023. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. In *ICLR*.

[24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR* (2024).

[25] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. RustBelt: Securing the Foundations of the Rust Programming Language. In *POPL*.

[26] Yoonseung Kim, Suhyun Lee, Youngjoon Kim, and Chung-Kil Hur. 2025. VeriRT: An End-to-End Verification Framework for Real-Time Distributed Systems. *POPL* 9 (2025), 1812–1839. https://doi.org/10.1145/3704897

[27] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *SOSP*.

[28] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. In *OOPSLA*.

[29] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *CACM* 52, 7. https://doi.org/10.1145/1538788.1538814

[30] Nathaniel Lewchenko, Gowtham Kaki, and Bor-Yuh Evan Chang. 2025. Bolt-On Strong Consistency: Specification, Implementation, and Verification. *OOPSLA* 9 (2025), 1604–1631. https://doi.org/10.1145/3720502

[31] Xinyao Luan, David Sanan, Zhe Hou, Qian Xu, Chunhui Liu, Yong Cai, Yang Liu, and Meng Sun. 2025. Why the Proof Fails in Different Versions of Theorem Provers: An Empirical Study of Compatibility Issues in Isabelle. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1499–1521. https://doi.org/10.1145/3715787

[32] Diego Ongaro. 2014. Consensus: Bridging Theory and Practice. https://web.stanford.edu/~ouster/cgi-bin/papers/OngaroPhD.pdf

[33] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm (Extended Version). *USENIX ATC* (2014). https://raft.github.io/raft.pdf

[34] George Pîrlea, Vladislav Gladshtein, Elan Kinsbruner, Qinshi Zhao, and Ilya Sergey. 2025. Veil: A Framework for Automated and Interactive Verification of Transition Systems. In *CAV*. 26–41. https://doi.org/10.1007/978-3-031-98682-6_2

[35] George Pîrlea and Ilya Sergey. 2018. Mechanising Blockchain Consensus. In *CPP*. 78–90. https://doi.org/10.1145/3167086

[36] Stanislas Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. 2022. Formal Mathematics Statement Curriculum Learning. *arXiv preprint arXiv:2202.01344* (2022).

[37] Lun Qiu, Yoonseung Kim, Jieung Shin, Joonseung Kim, Wolf Honoré, and Zhong Shao. 2024. LiDO: Linearizable Byzantine Distributed Objects with Refinement-Based Liveness Proofs. *PLDI* 8 (2024), 1140–1164. https://doi.org/10.1145/3656423

[38] Lun Qiu, Jie Xiao, Jiwon Shin, and Zhong Shao. 2025. LiDO-DAG: A Framework for Verifying Safety and Liveness of DAG-Based Consensus Protocols. *PLDI* 9 (2025), 1392–1416. https://doi.org/10.1145/3729306

[39] Muhammad Rashid, Iqra Rasool, Haroon Afzaal, and Nazir Ahmad Zafar. 2025. Formal verification of safety properties of epoch processing in Beacon Chain. *Scientific Reports* 15, 1 (2025). https://doi.org/10.1038/s41598-025-27396-w

[40] Talia Ringer, Karl Palmskog, Ilya Sergey, Milos Gligoric, and Zachary Tatlock. 2019. QED at Large: A Survey of Engineering of Formally Verified Software. *Foundations and Trends in Programming Languages* 5, 2–3 (2019), 102–281. https://doi.org/10.1561/2500000045

[41] Talia Ringer, RanDair Porter, Nathaniel Yazdani, John Leo, and Dan Grossman. 2021. Proof Repair across Type Equivalences. In *PLDI*. 112–127. https://doi.org/10.1145/3453483.3454033

[42] Mateo Sanabria, Leonardo Angel, and Nicolás Cardozo. 2024. Safety Verification of the Raft Leader Election Algorithm Using Athena. In *Advances in Computing*. 285–296. https://doi.org/10.1007/978-3-031-47372-2_23

[43] William Schultz, Ian Dardik, and Stavros Tripakis. 2022. Formal Verification of a Distributed Dynamic Reconfiguration Protocol. In *CPP*. 143–152. https://doi.org/10.1145/3497775.3503688

[44] Ilya Sergey, James R. Wilcox, and Zachary Tatlock. 2018. Programming and Proving with Distributed Protocols. *POPL* 2 (2018), 1–30. https://doi.org/10.1145/3158116

[45] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. 2024. Towards Large Language Models as Copilots for Theorem Proving in Lean. In *AI for Math Workshop, ICML*.

[46] Christoph Sprenger, Tobias Klenze, Marco Eilers, Felix Wolf, Peter Müller, Martin Clochard, and David Basin. 2020. Igloo: Soundly Linking Compositional Refinement and Separation Logic for Distributed System Verification. *OOPSLA* 4 (2020), 1–31. https://doi.org/10.1145/3428220

[47] The Aeneas Development Team. 2023. Charon: An LLBC Exporter for Rust. https://github.com/AeneasVerif/charon

[48] Amitayush Thakur, Yeming Wen, and Swarat Chaudhuri. 2024. An LLM Agent for Proving Theorems in Lean. In *NeurIPS Workshop on Math-AI*.

[49] Soren Eller Thomsen and Bas Spitters. 2021. Formalizing Nakamoto-Style Proof of Stake. In *CSF*. 1–15. https://doi.org/10.1109/CSF51468.2021.00042

[50] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. 2024. Solving Olympiad Geometry without Human Demonstrations. *Nature* 625 (2024), 476–482.

[51] Cosmo Viola, Muqiao Fan, and Talia Ringer. 2025. Proof Repair across Quotient Type Equivalences. *OOPSLA* 9 (2025), 3177–3202. https://doi.org/10.1145/3763164

[52] Haiming Wang, Huajian Xin, Chuanyang Zheng, Lin Li, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Lean Wang, Ruijia Wang, Haoran Wang, et al. 2024. LEGO-Prover: Neural Theorem Proving with Growing Libraries. In *ICLR*.

[53] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. 2015. Verdi: A Framework for Implementing and Formally Verifying Distributed Systems. In *PLDI*. ACM. https://doi.org/10.1145/2737924.2737958

[54] Doug Woos, James R. Wilcox, Steve Anton, Zachary Tatlock, Michael D. Ernst, and Thomas Anderson. 2016. Planning for Change in a Formal Verification of the Raft Consensus Protocol. In *CPP*. ACM. https://doi.org/10.1145/2854065.2854081

[55] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489* (2024).

[56] Kuo Xiong, Sunghwan Moon, Jeonghwan Kang, Benny Curto, Jihyun Kim, and Jiwon Shin. 2025. ReCraft: Self-Contained Split, Merge, and Membership Change of Raft Protocol. In *DSN*. 157–170. https://doi.org/10.1109/DSN64029.2025.00028

[57] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *NeurIPS* (2024).

[58] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Anima Anandkumar. 2024. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *NeurIPS*.

[59] Jianan Yao, Runzhou Tao, Ronghui Gu, and Jason Nieh. 2024. Mostly Automated Verification of Liveness Properties for Distributed Protocols with Ranking Functions. *POPL* 8 (2024), 1028–1059. https://doi.org/10.1145/3632877

[60] Travis Zhang, Travis Hance, Manos Kapritsos, Tej Chajed, and Bryan Parno. 2024. Inductive Invariants That Spark Joy. In *OSDI*. 837–853.

[61] Zhe Zhang, Ting Zhou, Christopher Jenkins, Omar Chowdhury, and Shuai Mu. 2025. AutoMan: Facilitating Verified Distributed Systems Development Through Automatic Code Generation and Manual Optimizations. In *SOSP*. 768–785. https://doi.org/10.1145/3731569.3764822

[62] Qinshi Zhao, George Pîrlea, Karolina Grzeszkiewicz, Seth Gilbert, and Ilya Sergey. 2024. Compositional Verification of Composite Byzantine Protocols. In *CCS*. 34–48. https://doi.org/10.1145/3658644.3690355